DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CSA06 - Design and Analysis of Algorithms

Assignment 3 - CO3

CO3 ASSIGNMENT REPORT

Divide and Conquer: Max/Min, Binary Search, Merge Sort, Quick Sort, Karatsuba, Strassen, Closest Pair, Median of Medians, Meet-in-the-Middle and the Day-4 Design Challenge

Item	Details
Course Outcome	CO3 - Design Divide and Conquer algorithms
Student A	Name:**shaik shahid** Reg. No.:**192424311**
Student B	Name**:Shaik Abdul Azeez** Reg. No.: **192472015)**
Group GitHub repository	**https://github.com/shaikshahid07/CSA0607-DAA**
Language and tools	Python 3, run in VS Code; NumPy and Matplotlib for experiments
Date of submission	2-11-2026

Overview and Method

This report answers the seven questions of the CO3 assignment. Every question has: the algorithm design and decomposition, pseudocode written before the implementation, the recurrence and its solution, test datasets, output screenshots, experimental results, comparison tables, graphs, observations and a technical conclusion. Both students worked on every question; the individual responsibilities are marked in each section (Student A / Student B) and summarised in the Contribution Record at the end.

How the experiments were run

- All programs are written in Python and print their results; the printed output was captured and is shown as the output screenshots.
- Random data uses a fixed seed (2026) so counts can be reproduced exactly. Timings depend on the computer, so they were averaged over several runs and are used only to compare methods and to find crossover ranges.
- Two kinds of evidence are used: **operation counts** (comparisons, multiplications, distance computations), which are exact and machine independent, and **running time**, which includes constant factors and overhead.

Summary of results

Q	Problem	Complexity achieved	Main observation
1	Max/Min; Binary Search	3n/2 - 2 comparisons; log2 n + 1	Exact match for n = 2^k; 1534 vs 2046 for n = 1024
2	Merge sort; Quick sort	Theta(n log n); n log n to n^2	Sorted input: depth 2047, 2.09 million comparisons vs 18,434 balanced
3	Karatsuba	Theta(n^1.585) vs n^2	Counts 12x lower at 1024 digits; time crossover about 128 to 192 digits
4	Strassen	Theta(n^2.807) vs n^3	Slower than conventional up to about 2048; crossover between 2048 and 4096; padding costly

Q	Problem	Complexity achieved	Main observation
5	Closest pair	Theta(n log n) vs n^2	5,118,400 vs 4,393 distance computations at n = 3200; D&C faster from about 16 points
6	Median of medians; Meet-in-the-middle	Theta(n) worst case; O(2^(n/2)) time and memory	Poor pivot: 24 million comparisons at n = 8000; MITM n = 40 with 2 million stored sums
7	Maximum subarray	Theta(n)	Exactly 5(n - 1) operations; doubling ratio 2.000

Q1 - Decomposition and Elimination in Ordered Data (6 marks)

1A. Finding Maximum and Minimum

Algorithm design and decomposition (Student A)

Decomposition. The list A[low..high] is split at mid = (low + high) / 2 into a left half and a right half. Each half is the same problem with about half the size. Splitting continues until a sub-list has one element (answer is the element itself, 0 comparisons) or two elements (one comparison orders them).

What each subproblem returns. A pair (max, min) for its part of the list.

Combine. The maximum of the whole list is the larger of the two maxima (1 comparison) and the minimum is the smaller of the two minima (1 comparison). So combining costs exactly 2 comparisons.

Decomposition for [8, 3, 15, 1, 9, 22, 4, 7]:

```
[8 3 15 1 9 22 4 7]
   [8 3 15 1]            [9 22 4 7]
   [8 3] [15 1]          [9 22] [4 7]
  (8,3) (15,1)          (22,9) (7,4)        <- 1
comparison each
   (15,1)                  (22,4)           <- 2
comparisons each
               (22,1)                       <- 2
comparisons
Total = 4 + 4 + 2 = 10 = 3(8)/2 - 2
```

Pseudocode

Part A - Finding Maximum and Minimum

Divide-and-Conquer idea: split the list into two halves, find (max, min) of each half, then combine with two comparisons.

```
MAXMIN(A, low, high)
    if low == high                              // one
element
        return (A[low], A[low])                 // 0
comparisons
    if high == low + 1                          // two
elements
        if A[low] > A[high]  return (A[low], A[high])
        else                 return (A[high], A[low])
// 1 comparison
    mid = (low + high) / 2
    (max1, min1) = MAXMIN(A, low, mid)              //
DIVIDE
    (max2, min2) = MAXMIN(A, mid + 1, high)         //
CONQUER
```

```
    max = larger of max1, max2                          //
COMBINE: 1 comparison
    min = smaller of min1, min2                         //
COMBINE: 1 comparison
    return (max, min)
```

- Base cases: size 1 (0 comparisons) and size 2 (1 comparison).
- Each subproblem returns a **pair** (max, min).
- Recurrence: T(n) = 2T(n/2) + 2, T(2) = 1 -> T(n) = 3n/2 - 2 (n = power of 2).
- Independent scan needs 2(n - 1) comparisons.

Recurrence and number of comparisons

T(n) = 2T(n/2) + 2 for n > 2, with T(2) = 1 and n a power of 2.

```
T(n) = 2T(n/2) + 2
Expand k-1 times (n = 2^k):
T(n) = 2^(k-1) * T(2) + 2 * (2^(k-1) - 1)
     = n/2 + (n - 2)
     = 3n/2 - 2
```

Comparison with the independent method. Finding max and min separately needs (n - 1) + (n - 1) = 2(n - 1) comparisons. The divide-and-conquer version needs 3n/2 - 2. The saving is n/2 comparisons, so the cost drops to about 75 percent for large n (for n = 1024: 1534 against 2046). Both are Theta(n); the improvement is only in the constant.

Experimental validation (Student B)

Datasets. (1) Random lists of sizes 2, 4, 8, ..., 16384. (2) Sizes that are not powers of two: 3, 5, 6, 7, 10, 25, 100, 333, 1000, 5000, 12345. (3) Four data patterns at n = 1024: random, ascending, descending, all equal. The code counts every comparison between list elements.

Q1 - python3 q1_maxmin_binary_search.py (part 1 of 2)

```
======================================================================
Q1 PART A : FINDING MAXIMUM AND MINIMUM (DIVIDE AND CONQUER)
======================================================================
Demo list      : [8, 3, 15, 1, 9, 22, 4, 7]
Max, Min       : 22 1
Comparisons    : 10

Experiment 1 : random data, sizes that are powers of two
n        Predicted    Observed     Straight     Saved      Correct
2        1            1            2            1          True
4        4            4            6            2          True
8        10           10           14           4          True
16       22           22           30           8          True
32       46           46           62           16         True
64       94           94           126          32         True
128      190          190          254          64         True
256      382          382          510          128        True
512      766          766          1022         256        True
1024     1534         1534         2046         512        True
2048     3070         3070         4094         1024       True
4096     6142         6142         8190         2048       True
8192     12286        12286        16382        4096       True
16384    24574        24574        32766        8192       True

Experiment 2 : sizes that are NOT powers of two
n        3n/2-2       Recurrence   Observed     Straight     Correct
3        3            3            3            4          True
5        6            6            6            8          True
6        7            8            8            10         True
7        9            9            9            12         True
10       13           14           14           18         True
25       36           39           39           48         True
100      148          162          162          198        True
333      498          536          536          664        True
1000     1498         1510         1510         1998       True
5000     7498         7950         7950         9998       True
12345    18516        20535        20535        24688      True
```

Output screenshot - Q1 (part 1 of 2)

```
Q1 - python3 q1_maxmin_binary_search.py   (part 2 of 2)

For n that is not a power of two the simple formula 3n/2-2 is only a lower estimate.
The exact recurrence T(n)=T(n/2)+T(n-n/2)+2 matches the observed count every time.

Experiment 3 : different kinds of data (n = 1024)
random       comparisons = 1534   (max=10234, min=5)
ascending    comparisons = 1534   (max=1023, min=0)
descending   comparisons = 1534   (max=1024, min=1)
all equal    comparisons = 1534   (max=5, min=5)

======================================================================
Q1 PART B : BINARY SEARCH
======================================================================
Sorted list    : [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]
Search 23   -> index 5    comparisons = 3
Search 2    -> index 0    comparisons = 3
Search 91   -> index 9    comparisons = 4
Search 40   -> index -1   comparisons = 4

Experiment 4 : successful and unsuccessful searches (every element tested)
n          log2(n)    Best case    Worst succ    Avg succ     Unsucc(max)
16         4          1            5             3.38         5
64         6          1            7             5.12         7
256        8          1            9             7.04         9
1024       10         1            11            9.01         11
4096       12         1            13            11.00        13
16384      14         1            15            13.00        15
65536      16         1            17            15.00        17
262144     18         1            19            17.00        19

Note: each loop pass counts as ONE comparison (the equality check at mid).
Worst case should be floor(log2 n) + 1 comparisons (log2 n + 1 when n = 2^k).

Experiment 5 : one million element array, a few searches
target 0         index 0        comparisons = 19
target 1000000   index 500000   comparisons = 19
target 1999998   index 999999   comparisons = 20
target 777       index -1       comparisons = 20
target -5        index -1       comparisons = 19
Linear search would need up to 1000000 comparisons.  log2(n) = 19.93
```

Output screenshot - Q1 (part 2 of 2)

Predicted against observed. For every power-of-two size the observed count equals 3n/2 - 2 exactly (for example 1, 4, 10, 22, ..., 24574 for n = 16384). The count is also identical for random, ascending, descending and all-equal data (1534 for n = 1024), because the algorithm never changes its comparisons based on the data values.

Discrepancy and explanation. For n that is not a power of two the observed count is higher than 3n/2 - 2 (n = 1000: observed 1510, formula 1498; n = 12345: 20535 against 18516). The formula was derived only for n = 2^k. For other n the halves have unequal sizes and some pieces reach size 1 (no comparison) while their partners reach size 2, so more combine steps happen per element. Solving the

exact recurrence T(n) = T(floor(n/2)) + T(ceil(n/2)) + 2 gives exactly the observed numbers (column 'Recurrence' in the output). The growth is still linear: for all sizes tested the count stays between 3n/2 - 2 and about 5n/3 - 2.

1B. Binary Search

Shared design

Part B - Binary Search

```
BINARYSEARCH(A, n, target)           // A is sorted
ascending
    low = 0, high = n - 1
    while low <= high
        mid = (low + high) / 2
        if A[mid] == target   return mid          //
found
        if A[mid] < target    low  = mid + 1      //
throw away left half and mid
        else                  high = mid - 1      //
throw away right half and mid
    return -1                                     //
search space empty
```

- Each pass throws away at least half of the remaining search space.
- Recurrence: T(n) = T(n/2) + 1 -> T(n) = floor(log2 n) + 1 comparisons.

Elimination logic (Student B)

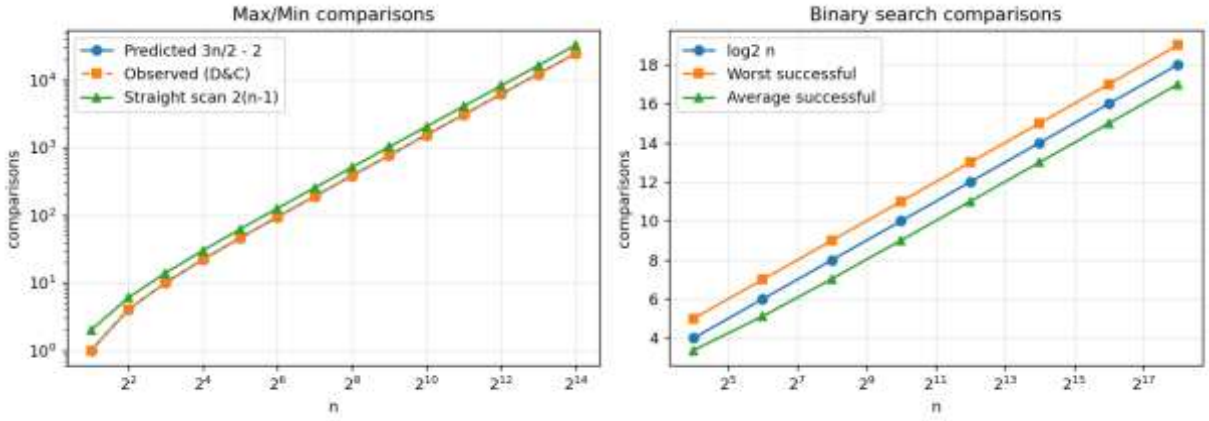
- **Decision rule.** Compare the target with A[mid]. Equal: done. A[mid] < target: continue in the right part. A[mid] > target: continue in the left part.
- **Eliminated portion.** If A[mid] < target, then mid and everything to its left are removed (at least half of the remaining elements). If A[mid] > target, mid and everything to its right are removed.
- **Why the target cannot be there.** The array is sorted ascending, so every element left of mid is <= A[mid]. If A[mid] < target, all of those are < target and cannot equal it. The symmetric argument holds for the right side when A[mid] > target.
- **Termination.** The search stops when the element is found or when low > high, which means the remaining search space is empty. The space size is roughly halved every pass (n, n/2, n/4, ..., 1, 0), so the loop ends after at most floor(log2 n) + 1 passes.
- **Efficiency.** Repeated halving is why the work is T(n) = T(n/2) + 1 = Theta(log n). One million elements need at most 20 comparisons; a linear scan needs up to 1,000,000.

Experimental investigation (Student A)

Cases. Every element of sorted arrays of size 2^4 to 2^18 was searched (successful search), and every value 'element + 1' (unsuccessful search, falls between two elements) plus a value below the array. One comparison is counted per loop pass (the check at mid).

n	log2 n	Best case	Worst successful	Average successful	Worst unsuccessful
16	4	1	5	3.38	5
256	8	1	9	7.04	9
4096	12	1	13	11.00	13
65536	16	1	17	15.00	17
262144	18	1	19	17.00	19

Evidence. The worst case is always log2 n + 1 and the average is about log2 n - 1. Doubling the data size adds only one comparison. For a one-million element array the program found items with 19 to 20 comparisons (see screenshot, Experiment 5).



Q1 graph comparisons

Cross-verification notes

Student A can explain the Max/Min decomposition and Student B can explain the Binary Search experiments: the core idea of both is to reduce the problem size by splitting it, Max/Min by splitting into two equal subproblems that are both solved, binary search by splitting into two and discarding one.

Conclusion

Divide and Conquer reduced Max/Min from 2(n - 1) to 3n/2 - 2 comparisons (observed exactly for powers of two) and Binary Search reaches a target among one million sorted items in at most 20 comparisons.

Q2 - The Quality of Decomposition in Sorting (7 marks)

2.1 Algorithms and pivot strategy

Merge Sort and Quick Sort were implemented. **Quick Sort pivot strategy: the first element of the (sub)list.** The list is partitioned into 'smaller than pivot' and 'greater than or equal to pivot' (this keeps the original order inside each part, which makes the balanced input easy to construct).

Merge Sort

```
MERGESORT(A)
    if length(A) <= 1  return A                          //
base case
    mid = length(A) / 2
    L = MERGESORT(A[0 .. mid-1])                         //
DIVIDE (always equal halves)
    R = MERGESORT(A[mid .. end])
    return MERGE(L, R)                                   //
COMBINE: linear work

MERGE(L, R)
    i = 0, j = 0, result = empty
    while i < |L| and j < |R|
        if L[i] <= R[j]  append L[i], i = i + 1
        else             append R[j], j = j + 1
    append the rest of L, then the rest of R
    return result
```

Recurrence: T(n) = 2T(n/2) + n -> Theta(n log n) for **every** input. Depth = ceil(log2 n) + 1.

Quick Sort - pivot strategy: FIRST element

```
QUICKSORT(A)
    if length(A) <= 1  return A
    pivot = A[0]
    left  = all x in A[1..] with x <  pivot              //
DIVIDE (n-1 comparisons)
    right = all x in A[1..] with x >= pivot
    return QUICKSORT(left) + [pivot] + QUICKSORT(right)
// COMBINE: just join
```

- Balanced input: pivot is the median at every level -> split n/2 : n/2 -> T(n) = 2T(n/2) + n.
- Unbalanced input (already sorted): left is empty at every level -> T(n) = T(n-1) + n -> O(n^2).

Building the experiment inputs

```
BUILDBALANCED(sorted S)                    // first element of
the list becomes the median
    if |S| <= 1  return S
    m = |S| / 2
    return [S[m]] + BUILDBALANCED(S[0..m-1]) +
BUILDBALANCED(S[m+1..])
```

Sorted and reverse-sorted arrays give the severely unbalanced case (1 : n-1) for the first-element pivot.

2.2 Quick Sort investigation (Student A)

- **Balanced input.** Built with BUILDBALANCED so the first element of every sublist is its median. For n = 15 the input is [8, 4, 2, 1, 3, 6, 5, 7, 12, 10, 9, 11, 14, 13, 15].
- **Severely unbalanced input.** An already sorted (or reverse sorted) list. The first element is the minimum (or maximum) every time.

Partition sizes recorded (n = 1023).

Depth	Balanced input (left, right)	Sorted input (left, right)
1	511, 511	0, 1022
2	255, 255	0, 1021
3	127, 127	0, 1020
4	63, 63	0, 1019
5	31, 31	0, 1018

Maximum recursion depth and comparisons (n = 2047).

Input	Quick Sort comparisons	Quick Sort max depth	Merge Sort comparisons / depth
Balanced	18,434	11	15,513 / 12
Random	25,755	23	19,944 / 12
Sorted	2,094,081	2047	11,253 / 12
Reversed	2,094,081	2047	11,263 / 12

How the pivot controls the decomposition. The pivot decides the sizes of the two subproblems. A pivot near the median gives two subproblems of size n/2, so the recursion tree is about log2 n levels deep (11 levels for n = 2047). A pivot that is the smallest element gives subproblems of size 0 and n - 1, so the tree becomes a chain

of n levels (2047). With the first-element pivot the input order decides the pivot, which is why sorted data, which looks 'easy', is the worst case.

```
Q2 - python3 q2_merge_quick_sort.py   (part 1 of 2)

======================================================================
Q2 : MERGE SORT AND QUICK SORT (pivot strategy = FIRST element)
======================================================================
Demo input        : [38, 27, 43, 3, 9, 82, 10]
Merge sort output : [3, 9, 10, 27, 38, 43, 82]  comparisons = 13  depth = 4
Quick sort output : [3, 9, 10, 27, 38, 43, 82]  comparisons = 13  depth = 5

Experiment 1 : first three partitions, n = 15
Balanced input  : [8, 4, 2, 1, 3, 6, 5, 7, 12, 10, 9, 11, 14, 13, 15]
   depth 1 : left = 7, right = 7
   depth 2 : left = 3, right = 3
   depth 3 : left = 1, right = 1
   depth 3 : left = 1, right = 1
   depth 2 : left = 3, right = 3
   depth 3 : left = 1, right = 1
   depth 3 : left = 1, right = 1
Sorted input    : [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]
   depth 1 : left = 0, right = 14
   depth 2 : left = 0, right = 13
   depth 3 : left = 0, right = 12
   depth 4 : left = 0, right = 11
   depth 5 : left = 0, right = 10
   depth 6 : left = 0, right = 9
   depth 7 : left = 0, right = 8

Experiment 2 : comparisons, depth and time for different inputs
n       Input        | QS comps   QS dep  QS time    | MS comps   MS dep  MS time
127     balanced     | 642        7       0.00007    | 584        8       0.00013
127     random       | 933        15      0.00013    | 722        8       0.00016
127     sorted       | 8001       127     0.00065    | 441        8       0.00012
127     reversed     | 8001       127     0.00054    | 447        8       0.00011
255     balanced     | 1538       8       0.00014    | 1364       9       0.00026
255     random       | 2218       17      0.00025    | 1744       9       0.00041
255     sorted       | 32385      255     0.00237    | 1016       9       0.00024
255     reversed     | 32385      255     0.00202    | 1023       9       0.00022
511     balanced     | 3586       9       0.00034    | 3113       10      0.00060
511     random       | 4301       16      0.00073    | 3960       10      0.00086
511     sorted       | 130305     511     0.00857    | 2295       10      0.00051
511     reversed     | 130305     511     0.00865    | 2303       10      0.00048
1023    balanced     | 8194       10      0.00067    | 6990       11      0.00130
1023    random       | 10553      19      0.00109    | 8931       11      0.00163
1023    sorted       | 522753     1023    0.03400    | 5110       11      0.00099
1023    reversed     | 522753     1023    0.03146    | 5119       11      0.00105
2047    balanced     | 18434      11      0.00149    | 15513      12      0.00268
2047    random       | 25755      23      0.00280    | 19944      12      0.00355
2047    sorted       | 2094081    2047    0.16149    | 11253      12      0.00217
2047    reversed     | 2094081    2047    0.12750    | 11263      12      0.00220
```

Output screenshot - Q2 (part 1 of 2)

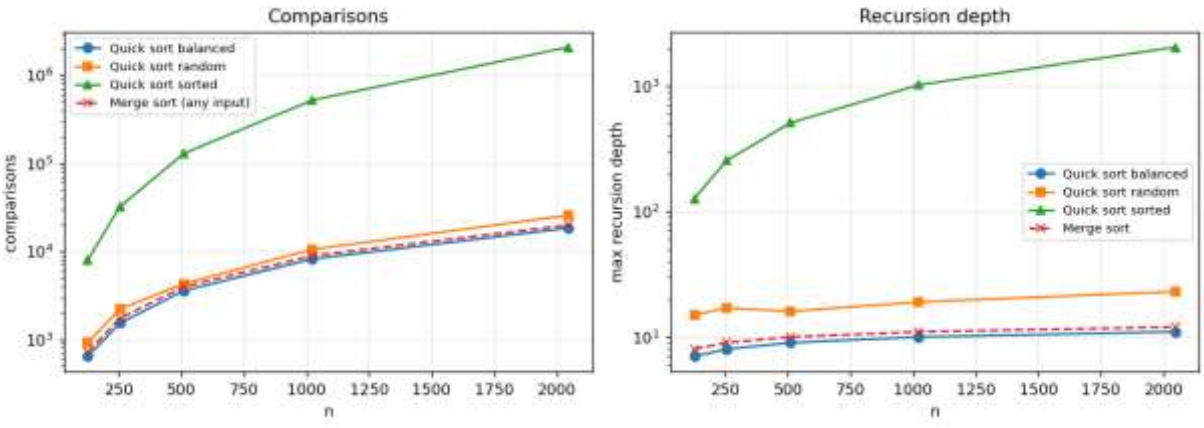
```
Q2 - python3 q2_merge_quick_sort.py   (part 2 of 2)

Experiment 3 : partition sizes for n = 1023 (first 5 partitions)
balanced
   depth 1 : left = 511, right = 511
   depth 2 : left = 255, right = 255
   depth 3 : left = 127, right = 127
   depth 4 : left = 63, right = 63
   depth 5 : left = 31, right = 31
sorted
   depth 1 : left = 0, right = 1022
   depth 2 : left = 0, right = 1021
   depth 3 : left = 0, right = 1020
   depth 4 : left = 0, right = 1019
   depth 5 : left = 0, right = 1018

Experiment 4 : height of the recursion tree for different split ratios (n = 1,000,000)
Split      Height formula         Height
50:50      log(n) / log(1/0.50)    19.9
25:75      log(n) / log(1/0.75)    48.0
10:90      log(n) / log(1/0.90)    131.1
1:99       log(n) / log(1/0.99)    1374.6
Totally unbalanced (1 : n-1) gives height n = 1000000

Experiment 5 : quick sort comparisons vs n^2/2 and n log2 n on sorted input
n        QS comps       n(n-1)/2       n log2 n
100      4950           4950           664
200      19900          19900          1529
400      79800          79800          3458
800      319600         319600         7715
1600     1279200        1279200        17030
```

Output screenshot - Q2 (part 2 of 2)



Q2 graph depth comparisons

2.3 Comparative D&C analysis (Student B)

- **Merge Sort decomposition.** Always splits at the middle, producing sizes floor(n/2) and ceil(n/2). The recursion tree height is ceil(log2 n) + 1 whatever the data looks like (observed 12 for n = 2047 in every row). The work is in the combine step: merge costs at most n - 1 comparisons per level, so T(n) = 2T(n/2) + n = Theta(n log n).

- **Quick Sort decomposition.** The split happens in the divide step (partitioning costs n - 1 comparisons) and the combine step is free. The sizes of the parts depend on the pivot.
- **Why Merge Sort is more predictable.** Its split does not depend on values, so its tree shape is fixed. Quick Sort's tree shape depends on data and pivot, so its cost ranges from n log n to n^2/2.
- **Partition quality and tree height.** If every split is p : (1 - p) the height is about log(n) / log(1 / (1 - p)) for p < 0.5. For n = 1,000,000: 50:50 gives about 20 levels, 25:75 about 48, 10:90 about 131, 1:99 about 1375, and a 1 : n-1 split gives 1,000,000 levels (Experiment 4).

2.4 Joint design question

If the input characteristics are not known in advance, what information would we want before choosing between Merge Sort and Quick Sort?

- **Is a worst-case guarantee required?** For real-time or adversarial settings choose Merge Sort (Theta(n log n) always), or Quick Sort with a better pivot rule (median of three, random) or Introsort.
- **Existing order.** Whether the data is nearly sorted, reverse sorted, or has many duplicates. A first-element pivot would be a bad choice for sorted data; a random or median-of-three pivot is safer.
- **Size of the data and memory.** Merge Sort needs O(n) extra space; in-place Quick Sort needs only O(log n) stack on average.
- **Stability.** If equal records must keep their relative order (sorting by several keys), Merge Sort is stable and standard Quick Sort is not.
- **Data structure and access pattern.** Merge Sort works well on linked lists and on external (disk) data because it reads sequentially; Quick Sort is cache-friendly on arrays.
- **How the data arrives.** Whether the input can be controlled by an attacker, and whether the whole input is available at once.

Decision. If these are unknown, a safe default is Merge Sort (or a hybrid such as Timsort/Introsort), because predictable decomposition protects against the quadratic worst case.

2.5 Conclusion

The quality of decomposition is the quality of the split. Balanced splits keep the recursion tree short (11 levels for 2047 elements); the unbalanced split made it 2047 levels and multiplied the comparisons by about 113 (2,094,081 against 18,434 for the balanced input).

Q3 - Breaking the Quadratic Barrier: Karatsuba Multiplication (7 marks)

3.1 Algorithmic derivation (Student A)

Decomposition of the integers. An n-digit number x is written with m = n/2 as x = xh * 10^m + xl, where xh is the upper half of the digits and xl the lower half. The same is done for y = yh * 10^m + yl. Example: 1234 = 12 * 10^2 + 34.

Ordinary expansion. x * y = xh*yh * 10^2m + (xh*yl + xl*yh) * 10^m + xl*yl. This needs four half-size multiplications, so T(n) = 4T(n/2) + cn = Theta(n^2), the same as the school method.

Karatsuba's idea. Let z2 = xh*yh, z0 = xl*yl and z1 = (xl + xh)(yl + yh) - z2 - z0. Expanding (xl + xh)(yl + yh) = xl*yl + xl*yh + xh*yl + xh*yh shows that z1 = xh*yl + xl*yh, which is exactly the middle term. The result is z2 * 10^2m + z1 * 10^m + z0. Only **three** half-size multiplications (z0, z2 and the sum product) are recursive. The rest are additions, subtractions and shifts, which cost O(n).

```
x = 1234, y = 5678, m = 2
xh = 12, xl = 34, yh = 56, yl = 78
z0 = 34*78 = 2652    z2 = 12*56 = 672
z1 = (34+12)(78+56) - 2652 - 672 = 46*134 - 3324 = 6164
- 3324 = 2840
result = 672*10^4 + 2840*10^2 + 2652 = 7006652
(matches 1234*5678)
```

Recurrence

T(n) = 3T(n/2) + cn, T(1) = constant.

```
Recursion tree: level i has 3^i subproblems of size
n/2^i, each costing c*n/2^i
Work at level i = 3^i * c * n / 2^i = c * n * (3/2)^i
Levels: i = 0 .. log2 n
Total = c*n * [ (3/2)^(log2 n + 1) - 1 ] / (3/2 - 1)
      = Theta( n * (3/2)^(log2 n) ) = Theta( 3^(log2 n)
) = Theta( n^(log2 3) )
Master theorem: a = 3, b = 2, f(n) = n = O(n^(log2 3 -
e)) -> Case 1 -> T(n) = Theta(n^1.585)
```

Result. Karatsuba is Theta(n^1.585) against Theta(n^2) for standard multiplication. For 1024-digit numbers the ideal ratio is 1024^2 / 3^10 = 17.8.

Pseudocode used

Standard (school-book) multiplication

```
STANDARD(x, y)
    a = digits of x, b = digits of y
    result = array of zeros, length |a| + |b|
    for i = 0 .. |a|-1
        carry = 0
        for j = 0 .. |b|-1
```

```
            total = result[i+j] + a[i]*b[j] + carry
            result[i+j] = total mod 10
            carry = total div 10
        result[i + |b|] = result[i + |b|] + carry
    return number formed by result          // n^2
single-digit multiplications
```

Karatsuba

```
KARATSUBA(x, y)
    if x < 10 or y < 10  return x * y                  //
base case: one digit
    n = max(digits of x, digits of y)
    m = n / 2
    p = 10^m
    x_high, x_low = x div p, x mod p                   //
DIVIDE each number in two halves
    y_high, y_low = y div p, y mod p
    z0 = KARATSUBA(x_low, y_low)                       //
3 recursive calls only
    z2 = KARATSUBA(x_high, y_high)
    z1 = KARATSUBA(x_low + x_high, y_low + y_high) - z0
- z2
    return z2 * p^2 + z1 * p + z0                      //
COMBINE: additions and shifts
```

Why it works: (xl + xh)(yl + yh) = xl*yl + (xl*yh + xh*yl) + xh*yh, so the middle term z1 = (xl*yh + xh*yl) is obtained from one multiplication instead of two.

Recurrence: T(n) = 3T(n/2) + cn -> Theta(n^(log2 3)) = Theta(n^1.585).

3.2 Crossover investigation (Student B)

- **Input sizes.** Random numbers with n = 4, 8, 16, 32, 48, 64, 96, 128, 192, 256, 384, 512, 768, 1024 digits (a finer step around the crossover).
- **Repetitions.** Each timing is the average of 7 runs on different random numbers. The same number pair is given to both methods.
- **Counts.** The code counts single-digit multiplications for both methods (standard: exactly n^2; Karatsuba: number of base-case products).

Q3 - python3 q3_karatsuba.py (part 1 of 2)

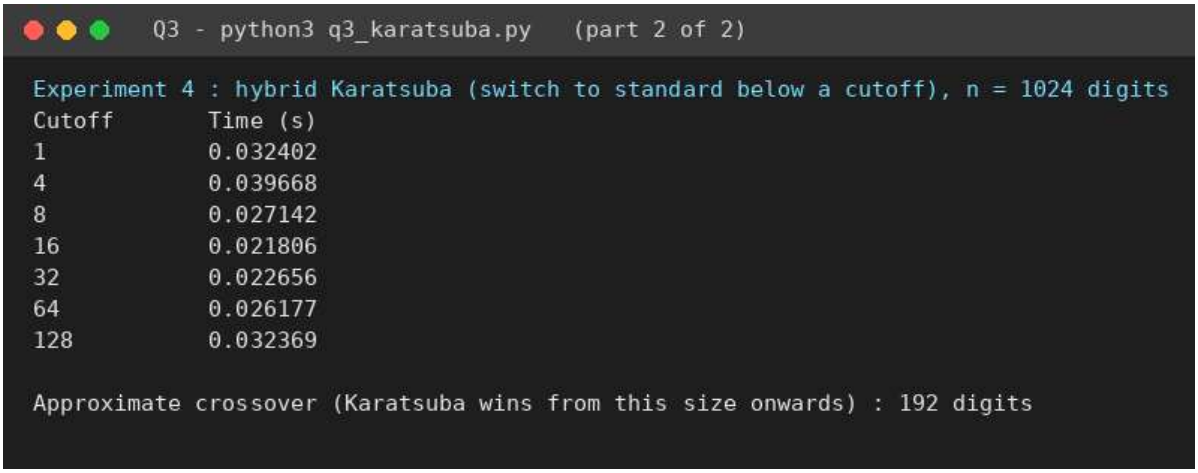
```
================================================================================
Q3 : KARATSUBA MULTIPLICATION VS STANDARD (SCHOOL-BOOK) MULTIPLICATION
================================================================================
Demo : 1234 x 5678
x = 12*10^2 + 34    y = 56*10^2 + 78
z0 = 2652   z2 = 672   z1 = 2840
z2*10^4 + z1*10^2 + z0 = 7006652
karatsuba  = 7006652  standard = 7006652  python = 7006652

Experiment 1 : correctness on 200 random pairs (1 to 300 digits)
Wrong answers : 0

Experiment 2 : single-digit multiplication counts (n-digit numbers)
n        Standard n^2   Karatsuba      n^1.585        Ratio
2        4              3              3              1.33
4        16             15             9              1.07
8        64             37             27             1.73
16       256            99             81             2.59
32       1024           329            243            3.11
64       4096           1073           729            3.82
128      16384          3297           2187           4.97
256      65536          9505           6561           6.89
512      262144         28611          19683          9.16
1024     1048576        86129          59049          12.17

Experiment 3 : timing (average of 7 runs, seconds)
n        Standard       Karatsuba      Faster
4        0.000007       0.000005       Karatsuba
8        0.000013       0.000017       Standard
16       0.000028       0.000042       Standard
32       0.000091       0.000136       Standard
48       0.000185       0.000245       Standard
64       0.000333       0.000426       Standard
96       0.000748       0.000850       Standard
128      0.001171       0.001186       Standard
192      0.004330       0.003770       Karatsuba
256      0.009197       0.007091       Karatsuba
384      0.022232       0.013406       Karatsuba
512      0.026317       0.011630       Karatsuba
768      0.059387       0.024527       Karatsuba
1024     0.115404       0.040196       Karatsuba
```

Output screenshot - Q3 (part 1 of 2)

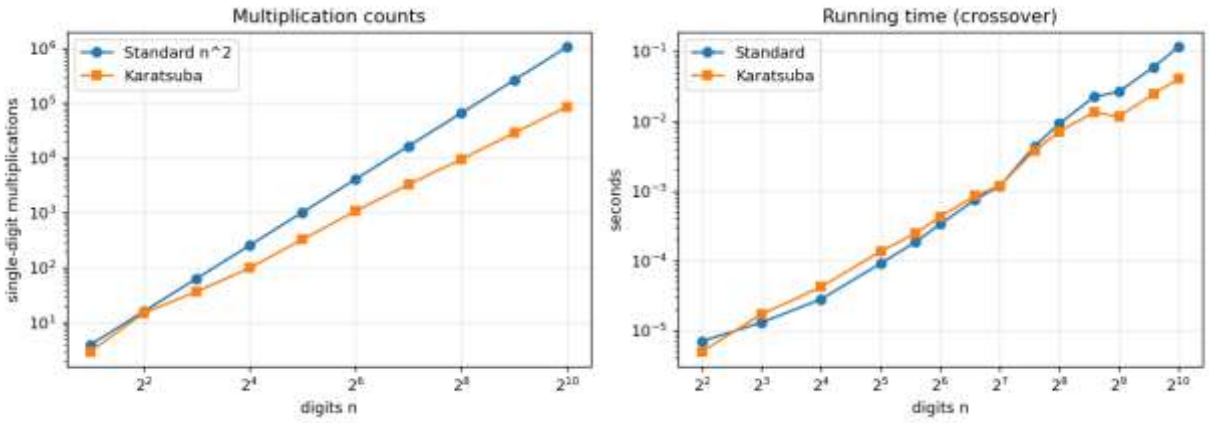


Output screenshot - Q3 (part 2 of 2)

n (digits)	Standard multiplications	Karatsuba multiplications	Ratio standard / Karatsuba
8	64	37	1.73
64	4,096	1,073	3.82
256	65,536	9,505	6.89
1024	1,048,576	86,129	12.17

Multiplication counts. Karatsuba always uses fewer single-digit multiplications for n >= 8, and the gap grows with n. Each doubling of n multiplies the Karatsuba count by about 3 (28,611 to 86,129 from 512 to 1024) and the standard count by 4. For tiny n (2 and 4 digits) the counts are almost equal because the sum (xl + xh) can have an extra digit, which creates larger subproblems than the n/2 assumed.

Approximate crossover range. In running time the standard method wins up to about 128 digits and Karatsuba wins from about 128 to 192 digits onward (it is slightly faster or about equal at 128 and clearly faster from 192; 192 was the first size from which it won in the final run). At 1024 digits Karatsuba is about 2.5 to 3 times faster. A hybrid that uses standard multiplication below about 16 to 32 digits was the fastest (21.8 ms against 32.4 ms for the pure version at n = 1024).



Q3 graph crossover

Why Karatsuba can be worse for small inputs. Each Karatsuba call performs three recursive calls, splits the numbers (division and modulus by 10^m), computes the digit lengths, adds the halves, subtracts z0 and z2, and combines with shifts. For small n these O(n) overheads and the function-call cost are larger than the multiplications saved. The standard method is a tight loop with no such overhead.

Recursion and decomposition overhead. The recursion tree has 3^(log2 n) leaves and about 1.5 times as many calls in total; each call allocates new numbers. The hybrid experiment confirms this: stopping the recursion at a cutoff of 16 to 32 digits reduced the time, while a large cutoff (128) made it slower again.

3.3 Joint conclusion

Why does better asymptotic complexity not automatically imply better performance for every input size? Big-O (or Theta) describes how the cost grows as n becomes large and hides constant factors and lower-order terms. The real cost is roughly c1 * n^1.585 for Karatsuba against c2 * n^2 for the standard method, with c1 much larger than c2 because of the extra additions, splitting and recursion. The two curves cross only where n^0.415 > c1 / c2. Below that crossover the method with the larger exponent but the smaller constant wins. Our measurements put that crossover at roughly 128 to 192 digits. Practical libraries therefore combine both methods.

Q4 - The Reality of Strassen's Matrix Multiplication (7 marks)

4.1 Algebraic design and correctness (Student B)

Conventional block decomposition. For n x n matrices (n a power of 2) write A and B as four n/2 x n/2 blocks. Then C11 = A11B11 + A12B21, C12 = A11B12 + A12B22, C21 = A21B11 + A22B21, C22 = A21B12 + A22B22. This needs **eight** block multiplications: A11B11, A12B21, A11B12, A12B22, A21B11, A22B21, A21B12, A22B22. T(n) = 8T(n/2) + Theta(n^2) = Theta(n^3).

Strassen's transformation (eight to seven). Seven products are formed from sums and differences of blocks:

```
M1 = (A11 + A22)(B11 + B22)        M5 = (A11 + A12) B22
M2 = (A21 + A22) B11               M6 = (A21 - A11)(B11 +
B12)
M3 = A11 (B12 - B22)               M7 = (A12 - A22)(B21 +
B22)
M4 = A22 (B21 - B11)
C11 = M1 + M4 - M5 + M7            C12 = M3 + M5
C21 = M2 + M4                      C22 = M1 - M2 + M3 + M6
```

Why the reconstructed blocks are correct.

```
C12 = M3 + M5 = A11B12 - A11B22 + A11B22 + A12B22 =
A11B12 + A12B22
C21 = M2 + M4 = A21B11 + A22B11 + A22B21 - A22B11 =
A21B11 + A22B21
C11 = M1 + M4 - M5 + M7
    = (A11B11 + A11B22 + A22B11 + A22B22) + (A22B21 -
A22B11)
      - (A11B22 + A12B22) + (A12B21 + A12B22 - A22B21 -
A22B22)
    = A11B11 + A12B21
C22 = M1 - M2 + M3 + M6
    = (A11B11 + A11B22 + A22B11 + A22B22) - (A21B11 +
A22B11)
      + (A11B12 - A11B22) + (A21B11 + A21B12 - A11B11 -
A11B12)
    = A22B22 + A21B12
```

Terms cancel exactly, so the four results equal the conventional blocks. The program also compares the Strassen result with the conventional result for every tested size (always equal).

Recurrence. T(n) = 7T(n/2) + 18(n/2)^2 (18 block additions or subtractions), so T(n) = Theta(n^(log2 7)) = Theta(n^2.807). The master theorem: a = 7, b = 2, f(n) = n^2 = O(n^(2.807 - e)), Case 1.

Padding. The recursion needs even sizes at every level, so a matrix of size n is copied into the top-left corner of a zero matrix of size 2^k >= n, multiplied, and the top-left n x n part of the result is taken. The extra zero rows and columns do not

change the answer (zero blocks produce zero products). The program tested n = 3, 5, 6, 10, 13, 20, 33 and the answers matched.

Cost of padding: the work grows by about (size / n)^2.807 for Strassen. Example: n = 33 pads to 64 (about 7.3 times the multiplications for the conventional method on the padded size), and n = 1025 pads to 2048.

Conventional block multiplication (8 multiplications)

```
BLOCKMULT(A, B)           // n x n, n a power of 2
    if n == 1  return A * B
    split A into A11 A12 / A21 A22 and B into B11 B12 /
B21 B22
    C11 = BLOCKMULT(A11,B11) + BLOCKMULT(A12,B21)
    C12 = BLOCKMULT(A11,B12) + BLOCKMULT(A12,B22)
    C21 = BLOCKMULT(A21,B11) + BLOCKMULT(A22,B21)
    C22 = BLOCKMULT(A21,B12) + BLOCKMULT(A22,B22)
    return joined C
```

T(n) = 8T(n/2) + O(n^2) -> Theta(n^3).

Strassen (7 multiplications)

```
STRASSEN(A, B)
    if n == 1  return A * B
    split A and B into four n/2 x n/2 blocks each
    M1 = STRASSEN(A11 + A22, B11 + B22)
    M2 = STRASSEN(A21 + A22, B11)
    M3 = STRASSEN(A11, B12 - B22)
    M4 = STRASSEN(A22, B21 - B11)
    M5 = STRASSEN(A11 + A12, B22)
    M6 = STRASSEN(A21 - A11, B11 + B12)
    M7 = STRASSEN(A12 - A22, B21 + B22)
    C11 = M1 + M4 - M5 + M7
    C12 = M3 + M5
    C21 = M2 + M4
    C22 = M1 - M2 + M3 + M6
    return joined C
```

T(n) = 7T(n/2) + 18(n/2)^2 -> Theta(n^(log2 7)) = Theta(n^2.807).

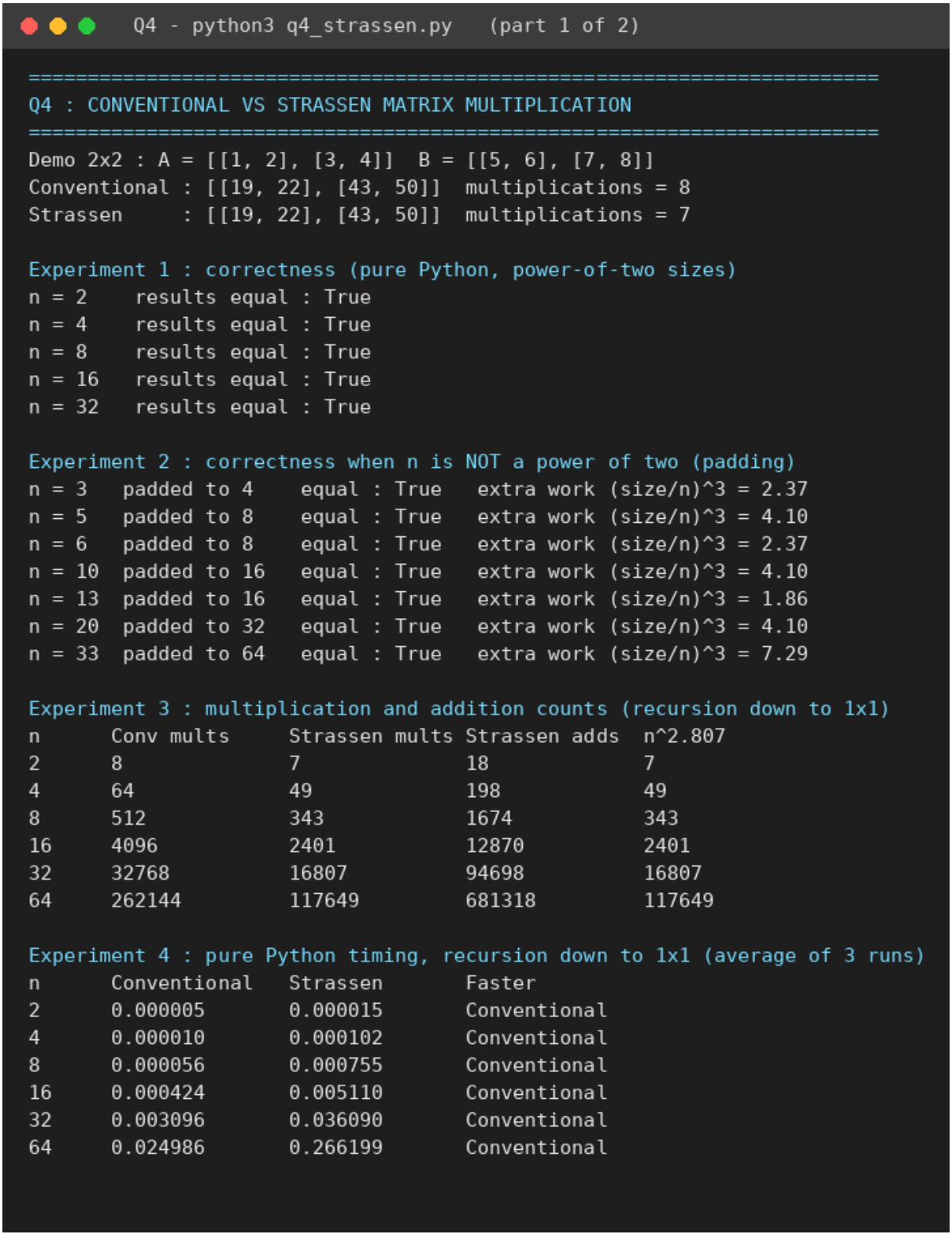
Padding when n is not a power of two

```
STRASSENANY(A, B)
    size = smallest power of 2 >= n
    PA, PB = A and B copied into size x size zero
matrices
    PC = STRASSEN(PA, PB)
    return top-left n x n part of PC
```

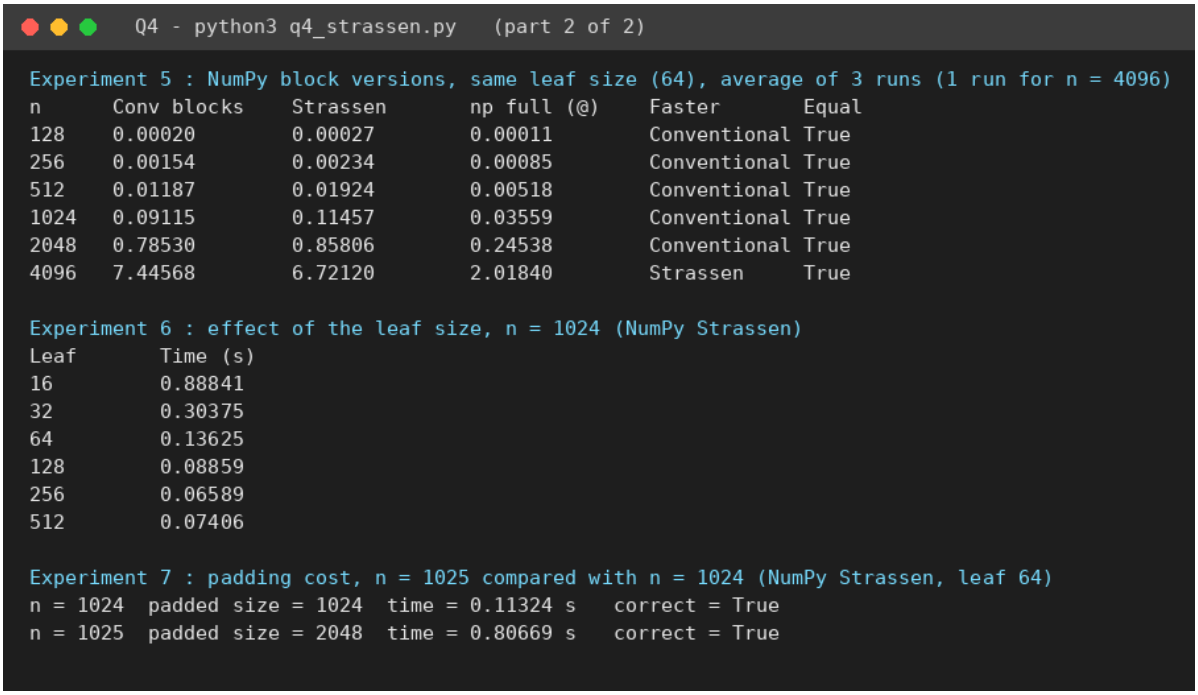
Extra cost: up to (size / n)^3 times more scalar multiplications for the conventional method (and (size/n)^2.807 for Strassen).

4.2 Experimental and engineering analysis (Student A)

- **Matrix sizes.** Pure Python: n = 2 to 64 (powers of 2). NumPy blocked versions: n = 128 to 4096. Padding test: n = 1024 and 1025.
- **Experiment design.** Pure Python conventional (triple loop) against pure Python Strassen recursing to 1 x 1. Then NumPy versions with the same recursion structure and the same 64 x 64 leaf size (conventional block recursion with 8 sub-products, Strassen with 7), plus NumPy's own A @ B (optimised BLAS) as a reference. Time averaged over 3 runs (1 run at n = 4096).
- **Repeatability.** Same random matrices for both methods; every result compared with the conventional result using allclose.



Output screenshot - Q4 (part 1 of 2)

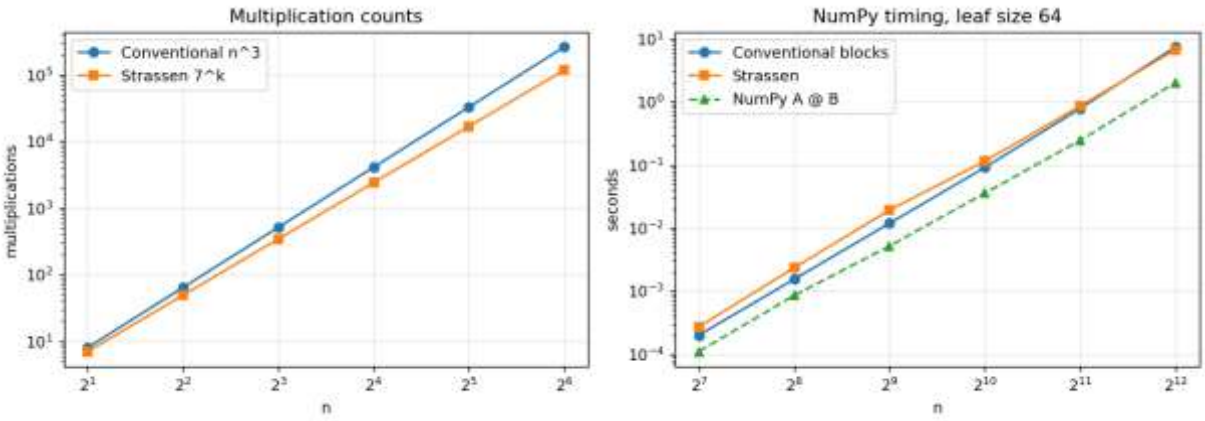


Output screenshot - Q4 (part 2 of 2)

n	Conventional multiplications	Strassen multiplications	Strassen additions	Ratio
8	512	343	1,674	1.49
16	4,096	2,401	12,870	1.71
32	32,768	16,807	94,698	1.95
64	262,144	117,649	681,318	2.23

Multiplication counts. Strassen's count is exactly 7^k (343, 2401, 16807, 117649) against 8^k, so the saving grows from 1.5 times at n = 8 to 2.2 times at n = 64. But Strassen needs many more additions (681,318 for n = 64, over five times the number of multiplications).

Practical crossover. In pure Python Strassen was slower at every size tested (n = 64: conventional about 0.03 s, Strassen about 0.27 s, about 10 times slower). Recursing to 1 x 1 creates 7^k function calls and many small lists. With the NumPy blocked versions and a leaf of 64, the conventional block version was faster for n = 128 to 2048 (for example n = 2048: 0.79 s against 0.86 s) and Strassen first won at n = 4096 (about 6.7 s against 7.4 s, a single run). The practical crossover for this machine and implementation is therefore between 2048 and 4096, and well above the matrix sizes of most applications.



Q4 graph strassen

Addition/subtraction overhead. Each Strassen level performs 18 matrix additions or subtractions of size n/2, each of which touches n^2/4 elements, and creates temporary matrices. These O(n^2) operations (memory-bound) cost much more per element than the multiply-add of a BLAS kernel.

Recursion overhead and leaf size. For n = 1024 the NumPy Strassen time fell from 0.89 s (leaf 16) to 0.31 s (leaf 32), 0.14 s (leaf 64), 0.09 s (leaf 128) and 0.07 s (leaf 256 to 512) as the recursion was cut earlier. Recursing all the way down is the slowest choice, so real implementations stop at a large leaf.

Padding overhead. n = 1024 took 0.11 s, but n = 1025 (padded to 2048) took 0.81 s, about 7 times longer for a 0.1 percent larger problem.

Memory and cache effects. Strassen allocates 7 product matrices and several temporary sums at each level, so its working memory is a constant multiple of n^2 more than the in-place conventional method. In every size, NumPy's A @ B (cache-blocked BLAS) was 2 to 4 times faster than our recursive versions, showing that cache-friendly access matters more than the operation count at these sizes.

4.3 Joint engineering decision

Under what conditions would we use conventional multiplication even though Strassen has better asymptotic complexity?

- **Small or medium matrices** (below roughly a few thousand rows in our tests): the crossover has not been reached.
- **An optimised library is available** (BLAS, GPU): hardware-tuned conventional kernels beat a hand-written Strassen by a large factor.
- **Numerical accuracy matters.** Strassen mixes sums and differences of blocks, so floating-point errors are larger than in the conventional method.
- **Memory is limited.** Strassen needs temporary matrices.
- **n is not a power of two** and padding would waste a lot of work (n = 1025 became 7 times slower).

- **Sparse or structured matrices**, where the conventional method can skip zeros.
- **Simplicity and maintainability** of the code.

Strassen becomes attractive for very large dense matrices with a tuned leaf size and enough memory.

Q5 - Upgrading Closest Pair Using Divide and Conquer (6 marks)

5.1 Decomposition design (Student A)

- **Organising the points.** The points are sorted once by x-coordinate. The recursion also returns the points sorted by y-coordinate (obtained by merging the two sorted halves, as in merge sort).
- **Dividing.** The x-sorted list is cut in the middle: the left half and right half each contain n/2 points and are separated by a vertical dividing line x = midx.
- **What each call returns.** The smallest distance in that part and that part's points sorted by y (so the parent can merge them).
- **Combine.** Let d = min(dL, dR). The only pairs not yet considered are pairs with one point in each half. Such a pair can only beat d if both points lie within distance d of the dividing line, so we take the 'strip' of points with |x - midx| < d, in y order, and check each point against the following points while their y difference is below d.
- **Why the boundary needs special treatment.** The recursive calls only look inside their own halves. A closest pair that straddles the line is invisible to both calls, so it must be looked for explicitly in the strip. If we skip it the answer can be wrong.

```
REC(Px):  if |Px| <= 3 brute force
          (dL, ...) = REC(left half)    (dR, ...) =
REC(right half)
          Py = merge both y-sorted lists   d = min(dL,
dR)
          strip = {p in Py : |p.x - midx| < d}
          for each p in strip, compare with next points
while y-gap < d
```

Recurrence. T(n) = 2T(n/2) + O(n) (merge by y + strip scan) = Theta(n log n), compared with Theta(n^2) for brute force. (If the strip were re-sorted at every call it would be Theta(n log^2 n); merging avoids that.)

5.2 Comparative evidence (Student B)

- **Equivalent datasets.** The same random point sets (unique integer points in a 1,000,000 x 1,000,000 square) were given to both algorithms, n = 50 to 6400; plus three layouts at n = 2000 (uniform, clustered, on a single line); correctness was checked on 300 random sets.
- **Recorded.** Number of distance computations (squared distances), running time and strip comparisons.

```
Q5 - python3 q5_closest_pair.py   (part 1 of 2)

======================================================================
Q5 : CLOSEST PAIR OF POINTS - BRUTE FORCE VS DIVIDE AND CONQUER
======================================================================
Demo points  : [(2, 3), (12, 30), (40, 50), (5, 1), (12, 10), (3, 4)]
Brute force  : squared distance 2 pair ((2, 3), (3, 4))  distance computations = 15
Divide&Conq  : squared distance 2 pair ((2, 3), (3, 4))  distance computations = 6

Experiment 1 : correctness on 300 random data sets (n between 2 and 200)
Wrong answers : 0

Experiment 2 : distance computations (uniform random points)
n       Brute dists    D&C dists      n log2 n     Ratio      t brute    t D&C
50      1225           60             282          20.4       0.0003     0.0001
100     4950           123            664          40.2       0.0010     0.0002
200     19900          258            1529         77.1       0.0039     0.0005
400     79800          519            3458         153.8      0.0164     0.0013
800     319600         1063           7715         300.7      0.0673     0.0025
1600    1279200        2138           17030        598.3      0.2854     0.0062
3200    5118400        4393           37260        1165.1     1.1509     0.0137
6400    20476800       8783           80921        2331.4     nan        0.0264
(for n = 6400 the brute force time is skipped; its count is n(n-1)/2)

Experiment 3 : different point layouts, n = 2000
Layout       Brute dists    D&C dists      Strip checks
uniform      1999000        2304           1232          same answer: True
clustered    1999000        2193           1121          same answer: True
on a line    1999000        1281           209           same answer: True

Experiment 4 : strip work per point (uniform random points)
n       Strip checks   Points         Checks per point
500     291            500            0.58
1000    585            1000           0.58
2000    1242           2000           0.62
4000    2519           4000           0.63
8000    5090           8000           0.64
```

Output screenshot - Q5 (part 1 of 2)

```
Q5 - python3 q5_closest_pair.py   (part 2 of 2)

Experiment 5 : timing in seconds, finding the practical crossover (average of 5 runs)
n       Brute        D&C          Faster
4       0.000002     0.000014     Brute
8       0.000007     0.000013     Brute
12      0.000015     0.000016     Brute
16      0.000030     0.000028     D&C
24      0.000058     0.000041     D&C
32      0.000104     0.000067     D&C
48      0.000231     0.000090     D&C
64      0.000404     0.000140     D&C
100     0.000970     0.000215     D&C
200     0.003900     0.000502     D&C
```

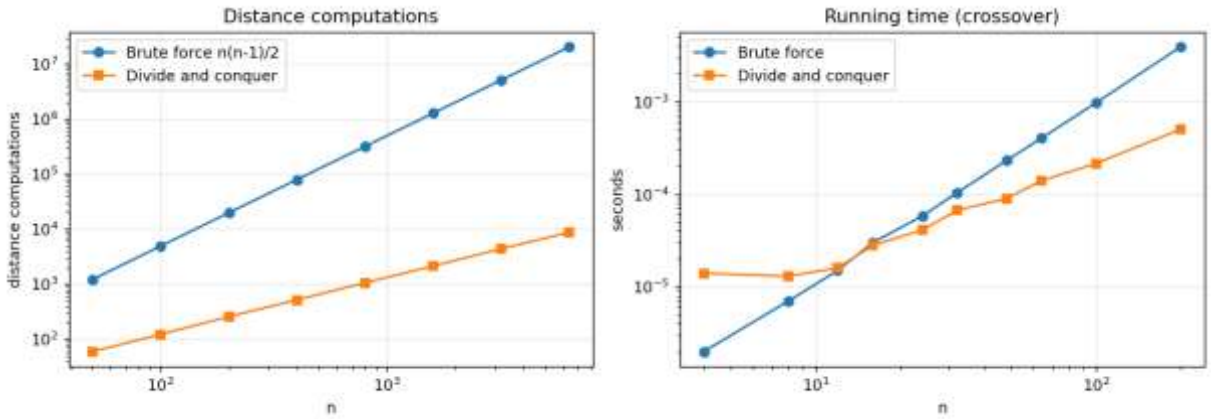
Output screenshot - Q5 (part 2 of 2)

n	Brute force distances	D&C distances	n log2 n	Ratio
100	4,950	123	664	40
400	79,800	519	3,458	154
1,600	1,279,200	2,138	17,030	598
3,200	5,118,400	4,393	37,260	1,165
6,400	20,476,800	8,783	80,921	2,331

Growth. When n doubles, brute force work grows about 4 times (1,279,200 to 5,118,400) while D&C work grows about 2 times (2,138 to 4,393). The D&C distance count stays close to 1.4 n (the n log n part of the cost is the sorting and merging, which are not distance computations). The saving ratio therefore keeps increasing with n.

Layouts. All three layouts (uniform, clustered, points on a line) gave the same answer as brute force, and D&C always used about 1300 to 2300 distance computations against 1,999,000 for brute force.

Practical crossover. Timing the two methods (average of 5 runs, repeated 20 times each) showed brute force is faster up to about 12 points and D&C is faster from about 16 points onwards (n = 100: 0.97 ms against 0.22 ms; n = 200: 3.9 ms against 0.5 ms). For lists of fewer than about 15 points the plain double loop is better, which is also why the D&C code uses brute force for 3 or fewer points.



Q5 graph closest pair

5.3 Cross-understanding

Why does the strip stage not require comparison with every other point?
(Student B) Only points within d of the dividing line can form a better pair, so most points are removed from the strip. Inside the strip the points are ordered by y, and a point only needs to be compared with later points whose y-difference is less than

d. All points that satisfy this lie in a d x 2d rectangle. The two d x d squares (left and right of the line) can each contain at most 4 points because every pair inside one half is at least d apart. So at most 8 points can be in the rectangle and each point needs at most 7 comparisons, a constant. Measured: only 0.58 to 0.64 strip comparisons per point for n = 500 to 8000 (Experiment 4).

What experimental evidence shows that the D&C strategy actually reduced work? (Student A) (1) The counted distance computations: 5,118,400 against 4,393 for n = 3200, with the ratio growing from 40 (n = 100) to 2,331 (n = 6400). (2) The brute force count follows n(n - 1)/2 exactly while the D&C count is near-linear (doubling n doubles it). (3) The running time: D&C is about 27 times faster at n = 800 and about 84 times at n = 3200, and its time grows almost proportionally to n. (4) Identical answers on 300 random tests, so the saving does not cost correctness.

5.4 Conclusion

Divide and Conquer upgraded closest pair from Theta(n^2) to Theta(n log n). The saving appeared in the counts for every n, and in real time from about 16 points.

Q6 - Advanced Divide-and-Conquer Mini-Investigations (7 marks)

6A. Median of Medians (Student A)

A. Median of Medians (k-th smallest)

```
SELECT(A, k)
    if |A| <= 5  return k-th smallest of A (sort)
    split A into groups of 5
    medians = median of each group                    //
insertion sort inside each group
    pivot = SELECT(medians, ceil(|medians| / 2))      //
median of the medians (recursion)
    low   = elements < pivot
    equal = elements == pivot
    high  = elements > pivot
    if k <= |low|                  return SELECT(low, k)
    if k <= |low| + |equal|        return pivot
    return SELECT(high, k - |low| - |equal|)
```

Guarantee: at least 3n/10 elements are <= pivot and at least 3n/10 are >= pivot, so the next call has at most 7n/10 elements.

Recurrence: T(n) = T(n/5) + T(7n/10) + O(n) -> Theta(n) because 1/5 + 7/10 < 1.

Grouping strategy. The list is cut into groups of 5 consecutive elements (the last group may be smaller). Each group is sorted with insertion sort (a constant cost for 5 elements) and its median is taken. Example from the program (n = 25): groups [3,38,42,45,55], [8,43,58,59,70], [1,5,15,84,95], [20,30,44,52,53], [21,64,88,94,97] have medians 42, 58, 15, 44, 88.

How the pivot is determined. The median of these n/5 medians is found by calling the same selection algorithm recursively (here 44). That value is used as the pivot.

Why consistently disastrous partitions are prevented. At least half of the group medians (n/10 of them) are <= pivot. Each such group has 2 more elements smaller than its median, so at least 3 * (n/10) = 3n/10 elements are <= pivot. In the same way at least 3n/10 are >= pivot. Therefore each recursive call after partitioning receives at most 7n/10 elements. In the 25-number example, 11 elements were smaller and 13 larger than the pivot (guarantee: at least 7.5 on each side).

Worst-case guarantee. T(n) <= T(n/5) + T(7n/10) + cn. Because 1/5 + 7/10 = 9/10 < 1, assume T(m) <= a*m for m < n. Then T(n) <= a*n/5 + 7a*n/10 + cn = (9a/10)n + cn <= a*n whenever a >= 10c. Hence T(n) = O(n) in the worst case.

Comparison with QuickSelect and the effect of a deterministic poor pivot

```
Q6A - python3 q6a_median_of_medians.py   (part 1 of 2)

================================================================
Q6 (A) : MEDIAN OF MEDIANS VS QUICKSELECT  (k-th smallest element)
================================================================
Demo list : [12, 3, 5, 7, 4, 19, 26, 1, 15, 9, 21, 8, 30, 2, 17]
Sorted    : [1, 2, 3, 4, 5, 7, 8, 9, 12, 15, 17, 19, 21, 26, 30]
k = 1    median-of-medians = 1    sorted[k-1] = 1
k = 4    median-of-medians = 4    sorted[k-1] = 4
k = 8    median-of-medians = 9    sorted[k-1] = 9
k = 15   median-of-medians = 30   sorted[k-1] = 30

Experiment 1 : correctness on 300 random lists with random k
Wrong answers : 0

Experiment 2 : grouping strategy and pivot for a list of 25 numbers
List    : [3, 38, 55, 45, 42, 8, 43, 58, 70, 59, 84, 1, 95, 5, 15, 20, 30, 53, 44, 52, 88, 97, 21, 64, 94]
Groups (each sorted) :
    [3, 38, 42, 45, 55]  median = 42
    [8, 43, 58, 59, 70]  median = 58
    [1, 5, 15, 84, 95]  median = 15
    [20, 30, 44, 52, 53]  median = 44
    [21, 64, 88, 94, 97]  median = 88
Medians : [42, 58, 15, 44, 88]  median of medians (pivot) = 44
Elements smaller than pivot : 11  larger : 13
Guarantee : at least 3n/10 = 7.5 elements on each side

Experiment 3 : comparisons on SORTED input, finding the median (k = n/2)
n       QS first       QS random      Median-of-med  n^2/2
500     93875          1903           2573           125000         True
1000    375250         3330           5311           500000         True
2000    1500500        6505           10951          2000000        True
4000    6001000        17466          22409          8000000        True
8000    24002000       29794          45464          32000000       True

Experiment 4 : steps (partition rounds) on sorted input, n = 8000, k = n/2
first    pivot : 4000 rounds
last     pivot : 4001 rounds
random   pivot : 16 rounds
```

Output screenshot - Q6A (part 1 of 2)

```
Q6A - python3 q6a_median_of_medians.py   (part 2 of 2)

Experiment 5 : randomised pivot is not perfect - 500 runs, n = 4000, sorted input, k = n/2
Best    : 6461
Average : 13503
Median  : 13175
Worst   : 28967
Linear guarantee of median-of-medians (cost, fixed every time) : 22409
Deterministic first-pivot cost (n^2/2 = 8,000,000)           : 6001000

Experiment 6 : random data, timing in seconds (k = n/2, average of 5 runs)
n       QS random      Median-of-med  sorted()
1000    0.00027        0.00086        0.00011
10000   0.00193        0.00911        0.00143
50000   0.01236        0.05034        0.01012
100000  0.03122        0.10528        0.02613
```

Output screenshot - Q6A (part 2 of 2)

n (sorted input)	QuickSelect, first pivot	QuickSelect, random pivot	Median of medians	n^2 / 2
500	93,875	1,903	2,573	125,000
1,000	375,250	3,330	5,311	500,000
2,000	1,500,500	6,505	10,951	2,000,000
4,000	6,001,000	17,466	22,409	8,000,000
8,000	24,002,000	29,794	45,464	32,000,000

- **Deterministic poor pivot.** Taking the first element on already sorted data removes one element per round: for n = 8000 it needed 4000 partition rounds and 24,002,000 comparisons (quadratic: 3 times more for 2 times the size is replaced by 4 times as n doubles). The median of medians needed 45,464 and doubled its work when n doubled (linear).
- **Randomisation reduces but does not eliminate poor partitions.** Over 500 random-pivot runs (n = 4000, sorted input, k = n/2) the cost ranged from 6,461 (best) to 28,967 (worst), average 13,503. The worst run was 29 percent above the guaranteed median-of-medians cost (22,409) and the average run was cheaper. A random pivot has a good chance (1/2) to land in the middle half of the data, but any single run can still draw a run of bad pivots; only the median-of-medians bound is certain.
- **Practical cost.** On random data median of medians was slower than random QuickSelect (n = 100,000: about 0.11 s against 0.03 s) because of its larger constant (grouping, extra recursion and extra passes). It is chosen when the worst-case guarantee matters.

Student B verification - why is pivot quality central to selection algorithms?
Selection discards the part of the list that cannot contain the k-th element, and the pivot decides how much is discarded. A pivot near the median removes about half of the data every round (total work n + n/2 + n/4 + ... = 2n, linear). A pivot near an end removes almost nothing (n + (n-1) + (n-2) + ... = quadratic). The whole algorithm is therefore only as good as its pivot rule.

6B. Meet-in-the-Middle Subset Sum (Student B)

B. Meet-in-the-Middle Subset Sum

```
SUBSETSUMS(list)
    sums = [0]
    for x in list  sums = sums + [s + x for s in sums]
    return sums                                          //
2^|list| values
```

```
MITM(nums, target)
    half = n / 2
    L = SUBSETSUMS(nums[0 .. half-1])                    //
2^(n/2) sums
    R = SUBSETSUMS(nums[half .. n-1])                    //
2^(n/2) sums
    sort L ascending, sort R ascending
    i = 0, j = |R| - 1
    while i < |L| and j >= 0
        total = L[i] + R[j]
        if total == target  return TRUE
        if total < target   i = i + 1
        else                j = j - 1
    return FALSE
```

Time O(2^(n/2) * n), memory O(2^(n/2)), compared with O(2^n) time for plain enumeration.

How the problem is divided. The n numbers are split into two halves of n/2 numbers. Any subset is a subset of the left half plus a subset of the right half.

Subset results for both halves. Each half generates all of its 2^(n/2) subset sums. For [3, 34, 4, 12] the sums are 0, 3, 4, 7, 12, 15, 16, 19, 34, 37, 38, 41, 46, 49, 50, 53; for [5, 2, 27, 8] they are 0, 2, 5, 7, 8, 10, 13, 15, 27, 29, 32, 34, 35, 37, 40, 42.

Combining. A subset of the whole list has sum = sl + sr, so we look for a left sum sl and right sum sr with sl + sr = target. Both lists are sorted and scanned with two pointers (smallest left with largest right): if the total is too small move the left pointer up, if too big move the right pointer down. This takes O(2^(n/2)) steps after sorting.

Search-space size. Instead of enumerating 2^n subsets, the algorithm creates 2 * 2^(n/2) sums. For n = 40: 1,099,511,627,776 against 2,097,152 (a reduction of 524,288 times). Time is O(2^(n/2) * n) because of generation and sorting.

Additional memory. Both sum lists are stored: 2 * 2^(n/2) numbers. Measured storage: n = 40 needs 2,097,152 stored sums (about 16.8 MB at 8 bytes each); n = 50 would need about 537 MB. Brute force can be run with O(n) memory if it does not store the sums, but our straightforward version stores all 2^n sums (n = 30 would need 8.6 GB), which is why it was only run up to n = 22.

```
Q6B - python3 q6b_meet_in_the_middle.py   (part 1 of 2)

================================================================
Q6 (B) : MEET-IN-THE-MIDDLE SUBSET SUM
================================================================
Demo numbers : [3, 34, 4, 12, 5, 2, 27, 8]
Left half    : [3, 34, 4, 12]  subset sums : [0, 3, 4, 7, 12, 15, 16, 19, 34, 37, 38, 41, 46, 49, 50, 53]
Right half   : [5, 2, 27, 8]  subset sums : [0, 2, 5, 7, 8, 10, 13, 15, 27, 29, 32, 34, 35, 37, 40, 42]
target 9    brute : True   MITM : True   subset = [4, 5]
target 30   brute : True   MITM : True   subset = [3, 27]
target 95   brute : True   MITM : True   subset = [3, 34, 4, 12, 5, 2, 27, 8]
target 96   brute : False  MITM : False  subset = None
target 100  brute : False  MITM : False  subset = None

Experiment 1 : correctness on 300 random cases (n between 2 and 16)
Wrong answers : 0

Experiment 2 : search-space size (worst case = target NOT reachable)
n     Brute 2^n        MITM 2*2^(n/2) sums      Reduction
10    1024             64                       16
16    65536            512                      128
20    1048576          2048                     512
24    16777216         8192                     2048
30    1073741824       65536                    16384
40    1099511627776    2097152                  524288
50    1125899906842624 67108864                 16777216
60    1152921504606846976 2147483648              536870912

Experiment 3 : running time and memory (target cannot be reached, so all sums are needed)
n     Brute time     MITM time      Brute stored     MITM stored
10    0.00004        0.00001        1024             64
14    0.00054        0.00003        16384            256
18    0.01441        0.00016        262144           1024
20    0.07616        0.00045        1048576          2048
22    0.31139        0.00062        4194304          4096
24    nan            0.00125        16777216         8192
28    nan            0.00535        268435456        32768
32    nan            0.02674        4294967296       131072
36    nan            0.13324        68719476736      524288
40    nan            0.75114        1099511627776    2097152
(brute force time is skipped for n above 22 : it would need 2^n sums in memory)
```

Output screenshot - Q6B (part 1 of 2)

```
Q6B - python3 q6b_meet_in_the_middle.py   (part 2 of 2)

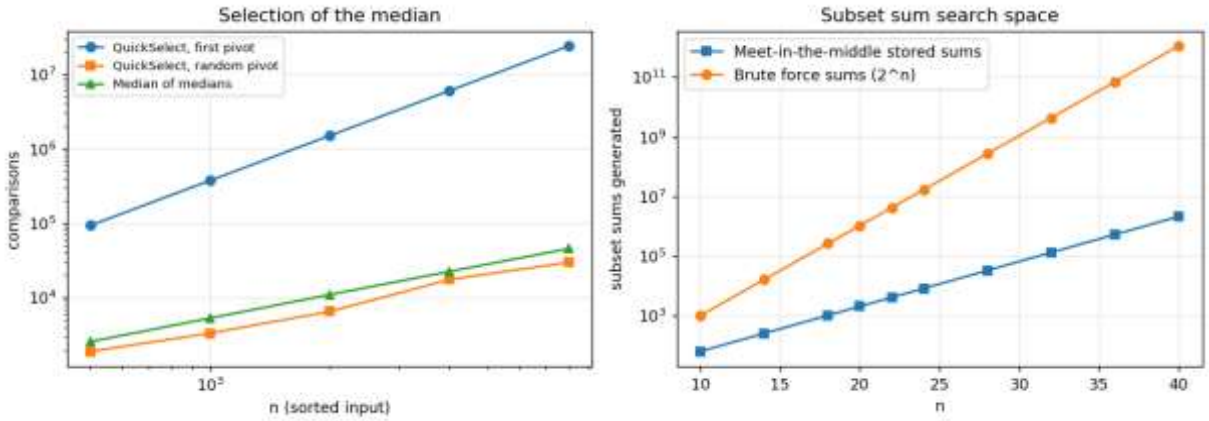
Experiment 4 : memory estimate for the stored lists (8 bytes per entry)
n     MITM memory        Brute memory
20    0.02 MB            8.39 MB
30    0.52 MB            8589.93 MB
40    16.78 MB           8796093.02 MB
50    536.87 MB          9007199254.74 MB

Experiment 5 : subset found when a solution exists (n = 30)
Target : 4406
Subset : [817, 352, 708, 243, 466, 745, 483, 9, 583]  sum = 4406
```

Output screenshot - Q6B (part 2 of 2)

n	Brute time (s)	MITM time (s)	Brute sums stored	MITM sums stored
18	0.014	0.0002	262,144	1,024

n	Brute time (s)	MITM time (s)	Brute sums stored	MITM sums stored
22	0.311	0.0006	4,194,304	4,096
28	not run	0.005	268,435,456	32,768
36	not run	0.13	68,719,476,736	524,288
40	not run	0.75	1,099,511,627,776	2,097,152



Q6 graph selection mitm

Time-memory trade-off. The algorithm spends memory (2^(n/2) stored sums) to avoid repeated work (2^n subsets). Square-rooting the time costs a square-root of the exponent in memory. If memory is reduced, the halves can be split into more pieces (for example the four-list Schroeppel-Shamir method uses about 2^(n/4) memory with the same 2^(n/2) time), or the method falls back to enumeration with small memory but 2^n time.

Student A verification - why can Meet-in-the-Middle reduce computational work even though it introduces additional storage? The two halves are independent, so each half can be enumerated separately with 2^(n/2) work, and the stored list of one half turns the 'find a matching partner' step into a sorted search instead of a fresh enumeration. The total work is 2^(n/2) + 2^(n/2) instead of 2^(n/2) x 2^(n/2) = 2^n, which is a far bigger saving than the storage (2^(n/2) numbers) costs, as long as that storage fits in memory.

Q7 - Day-4 Design Challenge: Divide and Conquer from First Principles (10 marks)

Problem used (assumption). Given an array A of n integers, find the largest sum of a contiguous non-empty subarray (Maximum Subarray Sum). If the Day-4 problem given in class was different, replace this chapter with the corresponding problem; the structure of the chapter is the same.

7.1 Stage 1 - Independent design proposals

Student A (initial design, retained in Q7_Day4_Design_Challenge/Student_A_Initial_Design): decompose into halves; base case one element; subproblem information = best sum inside the half; combine = max(best of left, best of right).

Student B (initial design): decompose into halves; base case one element; subproblem information = (total, best prefix, best suffix, best); combine using total_L + prefix_R and similar formulas.

(These two initial proposals are drafts in the repository and should be replaced by the pair's own original first ideas before submission.)

7.2 Stage 2 - Comparison of the two designs

Point	Student A	Student B
Decomposition	Two halves	Two halves (same)
Base case	One element	One element (same)
Information returned	1 value: best	4 values: total, prefix, suffix, best
Crossing subarray	Not handled: fails (demo array returns 4, correct 6)	Handled by suffix_L + prefix_R
Combine cost	O(1)	O(1)

Agree: the same divide step and base case. **Differ:** the information carried upward. **Missing cases found:** crossing subarray, all-negative array, odd length. **Retained:** the common decomposition and B's four-value record. Experimental result of the comparison: Student A's design gave the wrong answer in 1831 of 2000 random tests; B's gave 0 wrong answers.

7.3 Stage 3 - Joint final design

- **Decomposition:** split A[low..high] at the middle.
- **Subproblems:** left half and right half (same problem, half the size).
- **Base case:** a single element x returns (x, x, x, x).
- **Information returned:** (total, prefix, suffix, best).

- **Combine:** total = Ltotal + Rtotal; prefix = max(Lprefix, Ltotal + Rprefix); suffix = max(Rsuffix, Rtotal + Lsuffix); best = max(Lbest, Rbest, Lsuffix + Rprefix).

Problem used for the Day-4 challenge: given an array A of n integers (positive and negative), find the largest sum of a contiguous, non-empty subarray.

```
SOLVE(A, low, high)        // returns a record (total,
prefix, suffix, best)
    if low == high
        return (A[low], A[low], A[low], A[low])
// base case
    mid = (low + high) / 2
    L = SOLVE(A, low, mid)
// DIVIDE
    R = SOLVE(A, mid + 1, high)
// CONQUER
    total  = L.total + R.total
// COMBINE
    prefix = max(L.prefix, L.total + R.prefix)
    suffix = max(R.suffix, R.total + L.suffix)
    best   = max(L.best, R.best, L.suffix + R.prefix)
    return (total, prefix, suffix, best)

MAXSUBARRAY(A)
    return SOLVE(A, 0, n - 1).best
```

Meaning of each returned value for a segment:

- total = sum of the whole segment
- prefix = best sum of a subarray that starts at the left end of the segment
- suffix = best sum of a subarray that ends at the right end of the segment
- best = best sum of any subarray inside the segment

Recurrence: T(n) = 2T(n/2) + Theta(1) -> Theta(n).

Baselines used in the experiments

```
BRUTE(A)       for every i, for every j >= i, keep a
running sum and track the maximum         // Theta(n^2)
SCAN-DC(A)     best of left half, right half and
CROSSING(A, low, mid, high)                  // Theta(n
log n)
CROSSING       best sum going left from mid  +  best sum
going right from mid+1                 // Theta(n)
```

7.4 Stage 4 - Validation

Mathematical validation (Student A)

```
T(n) = 2T(n/2) + c          for n > 1,    T(1) = d
```

```
2T(n/2): the two recursive calls on the left and right
halves
c        : the combine step (4 additions/comparisons,
constant)

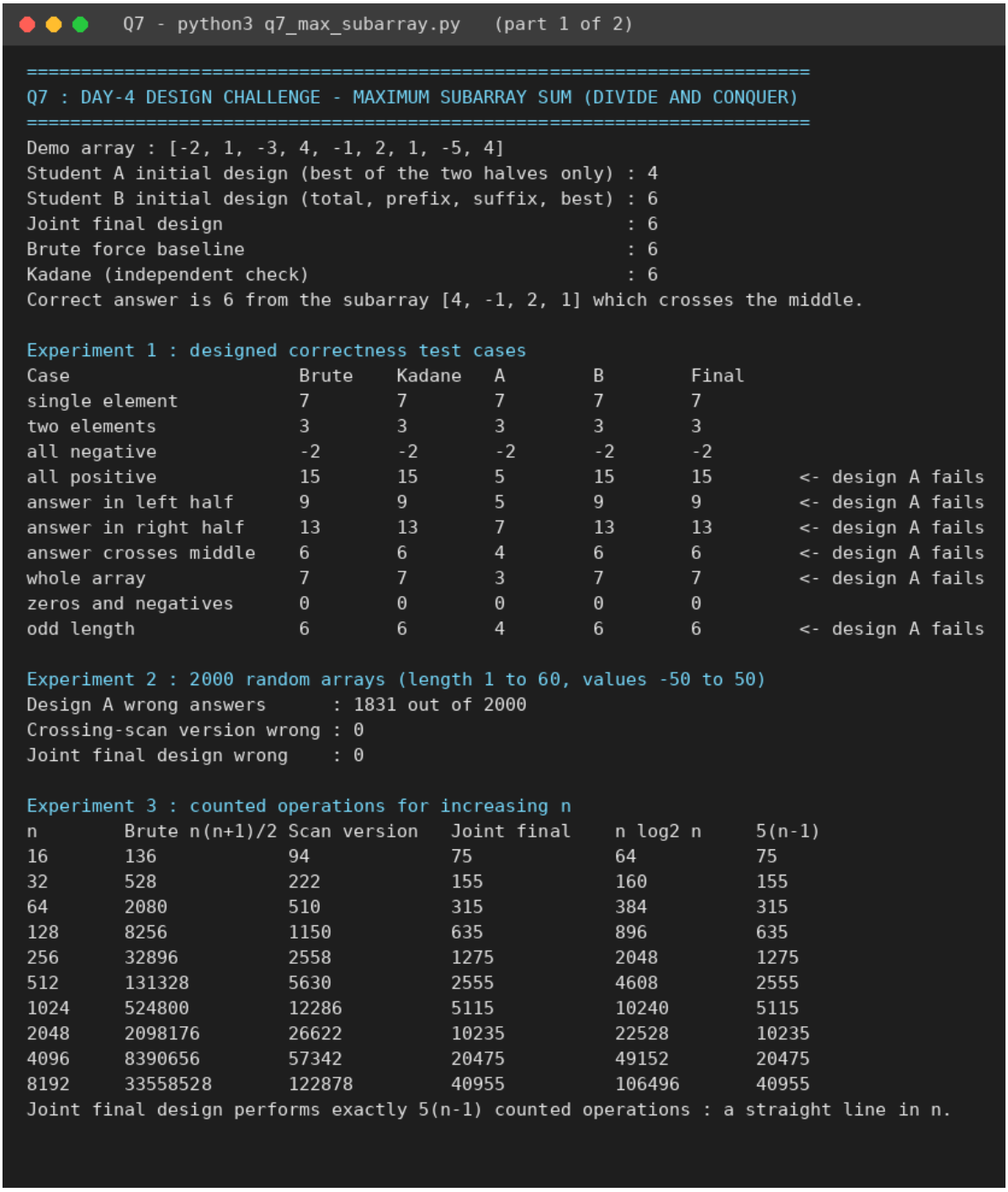
Recursion tree: level i has 2^i nodes, each costing c
Total combine cost = c(1 + 2 + 4 + ... + n/2) = c(n - 1)
Leaves: n * d
T(n) = c(n - 1) + d*n = Theta(n)

Master theorem: a = 2, b = 2, f(n) = Theta(1) = O(n^(1 -
e)) -> Case 1 -> Theta(n)
```

A version that keeps only 'best' and scans outward from the middle has combine cost Theta(n), so T(n) = 2T(n/2) + Theta(n) = Theta(n log n); the four-value design improves it to Theta(n).

Experimental validation (Student B)

- **Correctness test cases:** single element, two elements, all negative, all positive, answer in left half, answer in right half, answer crossing the middle, whole array, zeros and negatives, odd length; plus 2000 random arrays checked against brute force and Kadane's algorithm.
- **Baselines:** brute force Theta(n^2) with running sums, D&C with a crossing scan Theta(n log n), Kadane's linear scan.



Output screenshot - Q7 (part 1 of 2)

```
Q7 - python3 q7_max_subarray.py   (part 2 of 2)

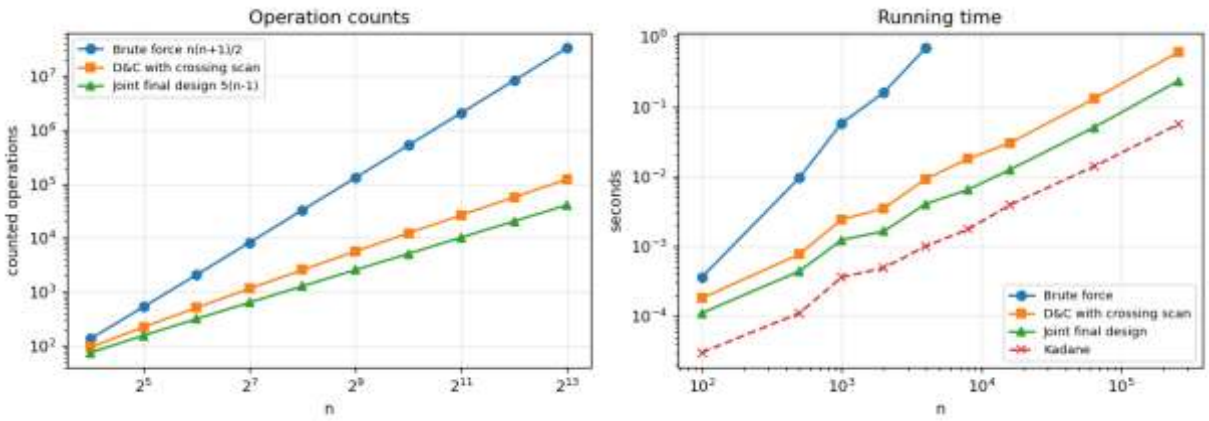
Experiment 4 : timing in seconds (average of 3 runs)
n        Brute        Scan D&C     Joint final  Kadane
100      0.00036      0.00018      0.00011      0.00003
500      0.00964      0.00077      0.00044      0.00011
1000     0.05800      0.00242      0.00123      0.00036
2000     0.15892      0.00347      0.00164      0.00049
4000     0.69360      0.00922      0.00404      0.00100
8000     skipped      0.01790      0.00645      0.00175
16000    skipped      0.03016      0.01253      0.00391
64000    skipped      0.13007      0.05055      0.01395
256000   skipped      0.60296      0.23384      0.05590

Experiment 5 : doubling test for the joint final design
n        Operations     Ratio
1024     5115           -
2048     10235          2.001
4096     20475          2.000
8192     40955          2.000
16384    81915          2.000
32768    163835         2.000
65536    327675         2.000
A ratio of about 2 when n doubles means linear growth, which matches T(n) = 2T(n/2) + O(1).
```

Output screenshot - Q7 (part 2 of 2)

n	Brute n(n+1)/2	D&C with scan	Joint final	5(n - 1)
16	136	94	75	75
256	32,896	2,558	1,275	1,275
1,024	524,800	12,286	5,115	5,115
8,192	33,558,528	122,878	40,955	40,955

Result. The joint design uses exactly 5(n - 1) counted operations, and the operation count ratio is 2.000 each time n doubles (linear growth). All answers agreed with brute force. In running time the brute force took about 0.69 s at n = 4000 while the joint design took about 0.004 s. Kadane's algorithm is faster still (about 0.001 s), which is a reminder that a simpler linear algorithm can beat a D&C one.



Q7 graph validation

7.5 Stage 5 - Cross-defence

Student A: what experimental evidence supports the group's design? (1) 10 designed test cases and 2000 random tests matched brute force and Kadane with zero errors, including the all-negative and crossing cases that broke the first design. (2) The operation count is exactly 5(n - 1) for every n tested, and doubling n doubles the count (ratio 2.000), which is what linear time predicts. (3) Against brute force the work was 524,800 against 5,115 operations at n = 1024, and the gap widens as n grows.

Student B: how does the recurrence arise from the group's decomposition? The array is split into two halves, each solved by the same function, which gives the 2T(n/2) term. Combining the two returned records takes a fixed number of additions and comparisons (4 operations plus the final max), independent of n, which gives the constant term c. The base case is a single element, T(1) = d. So T(n) = 2T(n/2) + c.

7.6 Stage 6 - Joint evaluation

- **Why the problem suits D&C:** the problem splits into two independent subproblems of the same kind; the answer for the whole depends on each half only through a small summary (4 numbers); the combine step is cheap.
- **Work the decomposition reduces:** brute force examines n(n+1)/2 subarrays. D&C never lists subarrays: it reuses the summaries of the halves, so each element is touched in a constant number of combine steps.
- **Additional costs of recursion:** about 2n function calls, a stack of depth log2 n, creation of a record at each call, and call overhead that makes the recursive version about 3 to 4 times slower than a simple loop in our timings.
- **When D&C might not be appropriate:** when subproblems overlap heavily (use dynamic programming), when the combine step costs as much as solving the whole problem (no reduction), when the split is very unbalanced, when n is tiny, or when a simpler linear algorithm exists (here Kadane).

What characteristics should an algorithm designer look for before choosing Divide and Conquer for a new problem?

1. The problem can be split into smaller subproblems of the **same type** with a clear base case.
2. The subproblems are **independent** (little or no overlap), so the same work is not repeated.
3. A small amount of **summary information** from the subproblems is enough to rebuild the answer.

4. The **combine step is cheaper** than solving the whole problem; the recurrence should show a real gain (a, b, f(n) check with the master theorem).
5. The split is reasonably **balanced**, or its quality can be controlled (pivot, median).
6. The extra cost of recursion (calls, stack and temporary memory) is acceptable for the input sizes, and a simple method is used for small inputs.
7. A well-suited simpler or better technique (greedy, dynamic programming) does not already exist.

Overall Technical Conclusions

- Divide and Conquer gets its power from the **recurrence**: the number of subproblems (a), the size reduction (b) and the combine cost f(n). Max/Min (a=2,b=2,f=O(1)), Karatsuba (a=3,b=2,f=n), Strassen (a=7,b=2,f=n^2) and the maximum subarray record (a=2,b=2,f=O(1)) each improve their problems by lowering a or f(n).
- **Balance matters.** Merge Sort's fixed halves gave depth 12 for 2047 elements for all inputs; Quick Sort ranged from depth 11 to depth 2047. Median of medians fixes the pivot problem and guarantees linear time.
- **Asymptotic improvement is not an immediate speed-up.** Karatsuba paid off from about 128 to 192 digits, Strassen only between n = 2048 and 4096, closest pair from about 16 points. Below these sizes constant factors and recursion overhead dominate, so practical programs use hybrids with a cutoff.
- **Time and memory trade.** Meet-in-the-Middle reduces 2^n to 2^(n/2) work at the cost of storing 2^(n/2) sums.
- **Choose D&C deliberately.** Look for same-type subproblems, independence, a cheap combine and a balanced split; do not use it when subproblems overlap or when a simpler linear method exists (Kadane for maximum subarray).

Student Contribution Record

The contribution record reflects the actual work completed. Primary contribution does not imply exclusive responsibility; both students remain accountable for the complete assignment.

Question	Student A - Primary Contribution	Student B - Primary Contribution	Joint Work
Q1	Max/Min decomposition drawing, combine step, derivation of 3n/2 - 2 and comparison with 2(n-1). Binary Search experiments: successful/unsuccessful cases, comparison counts, increasing sizes.	Max/Min experiments: datasets, comparison counting, predicted vs observed, explanation of discrepancy. Binary Search elimination logic, decision rule, termination and efficiency argument.	Understanding both problems, strategy, base cases, implementation and testing of both algorithms.
Q2	Quick Sort investigation: balanced and unbalanced inputs, partition sizes, maximum recursion depth, effect of pivot selection.	Merge Sort decomposition analysis, comparison with Quick Sort recursion, predictability, tree height, how imbalance gives O(n^2).	Merge/Quick Sort implementation, first-element pivot choice, experiments, joint design question (what to know about the input).
Q3	Integer decomposition, subproblem formation, why three multiplications, recurrence T(n) = 3T(n/2) + cn, solution Theta(n^1.585).	Crossover experiments: input sizes, repeated timing, multiplication counts, crossover range,	Standard and Karatsuba implementations, crossover experiment setup, joint conclusion on

Question	Student A - Primary Contribution	Student B - Primary Contribution	Joint Work
		small-input overhead.	asymptotic vs practical performance.
Q4	Experiments: matrix sizes, multiplication counts, repeated timing, practical crossover, addition/recursion/padding overhead, memory and cache effects.	Algebraic design: block decomposition, 8 vs 7 multiplications, correctness proof, padding.	Study of block multiplication, Strassen reconstruction, implementation and verification, joint engineering decision.
Q5	D&C decomposition design: organising points, dividing, what recursive calls return, combine step, boundary strip.	Comparative evidence: equivalent datasets, distance counts, execution behaviour, growth comparison, practical crossover.	Review of CO2 brute force, D&C design, implementation, comparison; cross-explanation of strip and evidence.
Q6	Median of Medians: implementation, grouping, pivot, worst-case guarantee, QuickSelect comparison, deterministic poor pivot, randomisation. Verification answer on Meet-in-the-Middle storage.	Meet-in-the-Middle: division, subset generation, combination, search space, memory, comparison with 2^n, time-memory trade-off. Verification answer on pivot quality.	Cross-understanding of both investigations.

Question	Student A - Primary Contribution	Student B - Primary Contribution	Joint Work
Q7	Independent initial design A, mathematical validation (recurrence and solution), cross-defence on experimental evidence.	Independent initial design B, experimental validation (test cases, instrumentation, baselines), cross-defence on how the recurrence arises.	Design comparison, joint final design, pseudocode, joint evaluation and conclusion.

Viva Preparation - Transfer Questions

If Quick Sort repeatedly produces a 90:10 split rather than a 50:50 split, what happens to the recursion structure? The tree is still a tree with a longest path of log(n) / log(10/9) levels, about 6.6 log2 n (131 levels for n = 1,000,000 against 20). Every level still costs about n, so the total remains O(n log n) with a larger constant; the quadratic behaviour appears only when splits keep removing a constant number of elements.

If available memory is reduced substantially, how would that affect the Meet-in-the-Middle decision? The stored sums 2 * 2^(n/2) may no longer fit (n = 50 would need about 537 MB). Options: split into four lists (Schroeppel-Shamir) for about 2^(n/4) memory at the same 2^(n/2) time, stream and sort in chunks, or fall back to O(n) memory enumeration with 2^n time if n is small.

If the matrix size is not a power of two, what does the Strassen implementation need to do, and what cost does this introduce? Pad both matrices with zeros up to the next power of two (or peel off odd rows and columns), multiply, and cut the result back. In our test n = 1025 became 2048 and the run time rose from 0.11 s to 0.81 s, about 7 times.

If the data in binary search is not sorted? The elimination argument fails (elements on the discarded side may equal the target), so the array must be sorted first (O(n log n)) or a linear search used.

If Karatsuba is given numbers of different lengths? The code uses n = max(digits) and splits both numbers at n/2 digits, so the shorter number simply has a smaller (or zero) high part; the answer stays correct.

Appendix A - Repository Structure

```
CO3_Assignment/
  Q1_MaxMin_Binary_Search/        Pseudocode
Implementation_Code  Output_Screenshots
  Q2_MergeSort_QuickSort/         Pseudocode
Implementation_Code  Output_Screenshots
  Q3_Karatsuba_Multiplication/    Pseudocode
Implementation_Code  Output_Screenshots
  Q4_Strassen_Matrix_Multiplication/  Pseudocode
Implementation_Code  Output_Screenshots
  Q5_Divide_Conquer_Closest_Pair/ Pseudocode
Implementation_Code  Output_Screenshots
  Q6_Advanced_Divide_Conquer/     Median_of_Medians
Meet_in_the_Middle  Pseudocode  Implementation_Code
Output_Screenshots
  Q7_Day4_Design_Challenge/
Student_A_Initial_Design  Student_B_Initial_Design
Joint_Final_Design  Pseudocode  Implementation_Code
Output_Screenshots
  Final_Report/
CO3_Assignment_Report.docx
Student_Contribution_Record.docx
  README.md
```